

Mobile Application Security Testing Requirements Document

Prepared By: Information Network Security Administration (INSA) 2025
Submitted by: [Client's Name / Organization Placeholder]
Submitted to: Information Network Security Administration (INSA)

Table of Contents

- 1. Introduction
 - 1. Purpose
 - 2. Ownership
 - 3. Scope
 - 2. Legal and Administrative Documents
 - 3. Technical Documentation for Web Application Security Testing
 - 1. Business Architecture and Design
 - 2. Features of the Web Application
 - 3. Security Functionality Document
 - 4. Secure Coding Standard Documentation
 - 5. Threat Modeling & Risk Management
 - 6. Previous Security Testing Reports
 - 4. API Security Audit Requirements
-

1. Introduction

Purpose

This document provides an overview of the Web and Mobile Application Security Testing Requirements Document by the Information Network Security Administration (INSA) for the **Deresegn** Point of Sale (POS) and SaaS Application, implementing the Ministry of Revenues (MoR) e-invoicing system.

Ownership

The owner of this document is the Deresegn Engineering Department, which is responsible for updating and maintaining this technical documentation.

Scope

The scope of this document is limited to detailing the technical architecture, security functionalities, and data flows of the Deresegn POS/SaaS application (Flutter Mobile/Web frontend, Laravel backend) to facilitate security audits by INSA.

2. Legal and Administrative Documents (Mandatory)

- 1. **Updated Trade License:** [To be provided]
 - 2. **TIN Number or National ID:** [To be provided]
 - 3. **System/Product Patent Certificate:** [To be provided if available]
-

3. Technical Documentation for Web Application Security Testing

3.1 Business Architecture and Design

Data Flow Diagram (DFD)

In the Deresegn application, data originates from the user via the Flutter mobile client (POS operations, settings, supplies). Authentication is passed to the Laravel backend via API securely (HTTPS). For MoR integration, the branch authenticates using a branch TIN and password, fetching a JWT. This is exchanged for a temporary MoR fiscal token. Invoice payloads are built in the POS app, sent to the backend for queueing/processing, and synchronized with the Ministry of Revenues. Successful processing generates an invoice PDF which can be printed locally or emailed. Data flows continuously monitor network connectivity; offline actions are queued securely on the local device (SQLite/Hive) and synced in the background when connectivity resumes.

System Architecture Diagram

The Deresegn Application is a cloud-based hybrid architecture:

- **Frontend:** Flutter-based mobile and web application. Supports Android, iOS, and Web. Handles local state, UI rendering, offline queues, and PDF generation.
- **Backend:** Laravel framework (PHP) RESTful API. Handles business logic, authentication, bulk invoice registration, user permissions, and integrations.
- **Database:** Relational Database (MySQL/PostgreSQL) managed by the Laravel backend (Eloquent ORM).
- **Integration:** Direct integration with the Ethiopian Ministry of Revenues (MoR) e-invoicing endpoints.
- **Infrastructure:** Application and Database hosted securely. Web traffic routed through secure HTTP (TLS 1.2+).

Entity Relationship Diagram (ERD) Overview

Core Entities:

- **User / Roles:** Defines permissions, user ID, secure hashed passwords.
- **Branch:** Specific branch TINs and configurations for decentralized auth.
- **Invoice / Receipt:** Tracks fiscal data, line items, totals, MoR status.
- **Supply / Item:** Inventory management.
- **Offline Queue:** Tracks locally stored operations pending synchronization.

3.2 Features of the Web Application

- **Development Frameworks:** Flutter (Frontend), Laravel / PHP (Backend).
- **Third-Party Integrations:** MoR E-Invoicing System, Cloudinary (for media uploads).
- **Actor/User Types:** Super Admin, Branch Manager, Cashier, User.
- **System Dependencies:** Mobile OS (Android/iOS) for the POS client, reliable server environment for Laravel API.
- **Security Standards:** TLS 1.2 for all communications, Token-based API Authentication (JWT/MoR tokens), Offline Data Integrity, Role-Based Access Control (RBAC).

3.3 Security Functionality Document

Authentication and Access Control

Deresegn utilizes Role-Based Access Control (RBAC) governed by the `RoleController` and `PermissionController`.

1. **Access Allocation:** Users are assigned roles based on job functions.
2. **API Authentication:** Carried out via JWT (JSON Web Tokens). MoR interactions utilize a specific two-step token exchange.
3. **Password Protection:** Passwords are mathematically hashed (e.g., PBKDF2/Bcrypt via Laravel's native hashing). Passwords are never stored in plaintext.

User ID Authentication

Unique User IDs and secure authentication are enforced for all users accessing the POS and backend APIs.

- Default administrative passwords are systematically changed post-deployment.
- Token scopes restrict what actions a client can perform.
- Application limits repeated access attempts by implementing API rate-limiting on login endpoints.

Complexity of Password & Expiry

- Passwords require sufficient length and complexity.
- Users can change passwords directly through the Profile/Settings screens.
- Session tokens expire, requiring the user to re-authenticate (or transparently refresh tokens) after prolonged idle periods.

Encryption & Secure Communication

- **Data in Transit:** All API requests, including MoR token exchanges and invoice submissions, are encrypted using HTTPS (TLS 1.2+), protecting against Man-in-the-Middle (MitM) attacks.
- **Data at Rest:** Sensitive local offline queues and user preferences are securely cached. Passwords are encrypted at rest on the backend database.

3.4 Secure Coding Standard Documentation

Secure Coding Requirements

- **Input Validation & Sanitization:** Laravel's built-in Form Requests validate all incoming data. Length, type, and format are checked before processing.

- **Authentication:** No hardcoded credentials. MoR credentials and backend secrets are managed via environment variables (.env).
- **Database Access:** Eloquent ORM prevents SQL injection by utilizing prepared statements.
- **Error Handling:** Detailed logs are stored server-side. Production environments do not disclose stack traces or SQL errors to the end-user.
- **Session Management:** JWT tokens are used, eliminating traditional session fixation vulnerabilities.

SAST / DAST Requirements

Code changes go through static analysis and dependency checking before deployment. No code is deployed to production without passing security reviews.

3.5 Threat Modeling & Risk Management

- **Asset Identification:** Critical assets include MoR fiscal tokens, user credentials, offline invoice queues, and financial records.
- **Vulnerability Assessment:** Addressed by enforcing TLS, token expiration, parameter validation, and secure local storage.
- **Offline Risk:** To mitigate tampering with offline queues, local data integrity checks are implemented before syncing to the backend.

3.6 Previous Security Testing Reports

Testing primarily utilizes automated source code analysis (Linting for Flutter, Static Analysis for PHP/Laravel) and dynamic manual testing (QA workflow) to ensure OWASP Top 10 vulnerabilities (like Injection, Broken Authentication, Sensitive Data Exposure) are mitigated.

4. API Security Audit Requirements

- **API Types:** RESTful JSON APIs (api.deresegn.com).
- **Authentication Mechanism:** JWT (JSON Web Tokens) for internal APIs, dynamic header injection for MoR fiscal tokens.
- **Third-Party Integrations:** Ethiopian Ministry of Revenues e-invoicing API.
- **Compliance:** Follows local data protection and financial regulatory requirements as mandated by MoR.
- **Authorization:** Endpoint-level middleware ensures only authorized roles can invoke specific controllers (e.g., BulkInvoiceRegistration, RoleController, etc.).

Note: As per the INSA guidelines, the application binary (APK/IPA) and the source code modules required for static analysis will be provided securely upon request via CD/DVD or the INSA Audit Request Portal.